

AI Assisted Coding Lab ASS-11.1

Name: B . SATHWIK REDDY

Batch:14

2303A510B1

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

PROMPT:

#Generate a clean, well-documented Python Stack class using a list with push, pop, peek, and is_empty methods, proper error handling, comments, and example usage.

This screenshot shows the Visual Studio Code editor with a Python file named 11.1.1.py. The code defines a Stack class using a list. The class has methods for push, pop, peek, and is_empty. The push method adds an item to the top of the stack. The pop method removes and returns the item at the top of the stack, raising an IndexError if the stack is empty. The peek method returns the item at the top of the stack without removing it, also raising an IndexError if the stack is empty. The is_empty method checks if the stack is empty. The class is initialized with an empty list. The code is as follows:

```
1 #Generate a clean, well-documented Python Stack class using a list with push, pop, peek, and is_empty methods, proper error ha
2 class Stack:
3     """
4     A simple Stack implementation using a list.
5
6     Methods:
7         push(item): Add an item to the top of the stack.
8         pop(): Remove and return the item at the top of the stack.
9         peek(): Return the item at the top of the stack without removing it.
10        is_empty(): Check if the stack is empty.
11    """
12
13    def __init__(self):
14        """Initialize an empty stack."""
15        self.stack = []
16
17    def push(self, item):
18        """Add an item to the top of the stack."""
19        self.stack.append(item)
20
21    def pop(self):
22        """
23        Remove and return the item at the top of the stack.
24
25        Raises:
26            IndexError: If the stack is empty.
27        """
28        if self.is_empty():
29            raise IndexError("Stack is empty. Cannot pop from an empty stack.")
30        return self.stack.pop()
31
32    def peek(self):
33        """
34        Return the item at the top of the stack without removing it.
35
36        Raises:
37            IndexError: If the stack is empty.
```

This screenshot shows the Visual Studio Code editor with the same Python file, 11.1.1.py. The code continues with a main function that demonstrates the usage of the Stack class. It pushes three items (1, 2, 3) onto the stack, prints the top item (3), pops it, and prints the top item again (2). It also checks if the stack is empty (False) and if it is empty after popping (True). Finally, it attempts to pop from an empty stack and peek at the top of an empty stack, both of which raise an IndexError. The code is as follows:

```
32 class Stack:
33     def peek(self):
34         """
35         IndexError: If the stack is empty.
36         """
37         if self.is_empty():
38             raise IndexError("Stack is empty. Cannot peek from an empty stack.")
39         return self.stack[-1]
40
41     def is_empty(self):
42         """check if the stack is empty."""
43         return len(self.stack) == 0
44
45 if __name__ == "__main__":
46     # Example usage of the Stack class
47     stack = Stack()
48
49     stack.push(1)
50     stack.push(2)
51     stack.push(3)
52
53     print("Top item (peek):", stack.peek()) # Output: 3
54     print("Popped item:", stack.pop()) # Output: 3
55     print("Is stack empty?", stack.is_empty()) # Output: False
56
57     stack.pop()
58     stack.pop()
59
60     print("Is stack empty?", stack.is_empty()) # Output: True
61
62     try:
63         stack.pop() # This will raise an error since the stack is empty
64     except IndexError as e:
65         print("Error:", e)
66
67     try:
68         stack.peek() # This will also raise an error since the stack is empty
69     except IndexError as e:
70         print("Error:", e)
71
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\sathw> c:; cd 'c:\Users\sathw'; & 'c:\Users\sathw\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\sathw\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62491' '--' 'C:\Users\sathw\11.1.1.py'
Top item (peek): 3
Top item (peek): 3
Popped item: 3
Is stack empty? False
Is stack empty? True
Error: Stack is empty. Cannot pop from an empty stack.
Is stack empty? True
Error: Stack is empty. Cannot pop from an empty stack.
Error: Stack is empty. Cannot pop from an empty stack.
Error: Stack is empty. Cannot peek from an empty stack.
PS C:\Users\sathw>
```

Task 1 – Stack

Justification:

Stack follows the LIFO principle, where the last inserted element is removed first. It is useful in undo operations, recursion, and expression evaluation.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

PROMPT:

Create a FIFO-based Python Queue class using a list with enqueue, dequeue, peek, and size methods, including docstrings, comments, and example usage.

```
16-1.py 16-2.py 8.1.py python 8.1.py 19-02-26.py 10.4-2.py 11.1.1.py X
C:\Users\sathw > 11.1.1.py > ...
1  #Create a FIFO-based Python Queue class using a list with enqueue, dequeue, peek, and size methods, including docstrings, comments, and type hints.
2  class Queue:
3      """
4      A simple implementation of a FIFO (First In, First Out) Queue using a list.
5
6      Methods:
7      - enqueue(item): Adds an item to the end of the queue.
8      - dequeue(): Removes and returns the item at the front of the queue. Raises an exception if the queue is empty.
9      - peek(): Returns the item at the front of the queue without removing it. Raises an exception if the queue is empty.
10     - size(): Returns the number of items in the queue.
11     """
12
13     def __init__(self):
14         """Initialize an empty queue."""
15         self.items = []
16
17     def enqueue(self, item):
18         """Add an item to the end of the queue."""
19         self.items.append(item)
20
21     def dequeue(self):
22         """Remove and return the item at the front of the queue. Raises an exception if the queue is empty."""
23         if self.size() == 0:
24             raise IndexError("Dequeue from an empty queue")
25         return self.items.pop(0)
26
27     def peek(self):
28         """Return the item at the front of the queue without removing it. Raises an exception if the queue is empty."""
29         if self.size() == 0:
30             raise IndexError("Peek from an empty queue")
31         return self.items[0]
32
33     def size(self):
34         """Return the number of items in the queue."""
35         return len(self.items)
36
37 # Example usage
38 if __name__ == "__main__":
39     queue = Queue()
40     queue.enqueue(1)
41     queue.enqueue(2)
42     queue.enqueue(3)
43
44     print("Queue size:", queue.size())
45     print("Front item:", queue.peek())
46
47     print("Dequeue item:", queue.dequeue())
48     print("Queue size after dequeue:", queue.size())
49     print("Front item after dequeue:", queue.peek())
50     queue.dequeue()
51     queue.dequeue()
52     print("Queue size after dequeuing all items:", queue.size())
53
54
55
56
57
58
59
```

OUTPUT:

```
Queue size: 3  
Front item: 1  
Dequeue item: 1  
Queue size after dequeue: 2  
Front item after dequeue: 2  
Queue size after dequeuing all items: 0  
PS C:\Users\sathw>
```

Task 2 – Queue

Justification:

Queue follows the FIFO principle, where the first inserted element is removed first. It is commonly used in scheduling and real-time processing systems.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
pass
```

```
class LinkedList:
```

```
pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

PROMPT:

#Generate a Python implementation of a Singly Linked List with Node and LinkedList classes including insert and display methods, proper documentation, comments, and example usage.

```
16-1.py 16-2.py 8.1.py python 8.1.py 19-02-26.py 10.4-2.py 11.1.1.py X
C:\> Users \sathw > 11.1.1.py > ...
1 #Generate a Python implementation of a Singly Linked List with Node and LinkedList classes including insert and display method
2 class Node:
3     """
4     A class representing a node in a singly linked list.
5
6     Attributes:
7     | data: The value stored in the node.
8     | next: A reference to the next node in the list.
9     """
10    def __init__(self, data):
11        """
12        Initializes a new node with the given data and sets the next reference to None.
13
14        Args:
15        | data: The value to be stored in the node.
16        """
17        self.data = data
18        self.next = None
19    class LinkedList:
20        """
21        A class representing a singly linked list.
22
23        Attributes:
24        | head: The first node in the linked list.
25        """
26        def __init__(self):
27            """
28            Initializes an empty linked list by setting the head to None.
29            """
30            self.head = None
31
32        def insert(self, data):
33            """
34            Inserts a new node with the given data at the end of the linked list.
35
36            Args:
```

```
37            | data: The value to be stored in the new node.
38            """
39            new_node = Node(data)
40
41            if self.head is None:
42                self.head = new_node
43                return
44
45            last_node = self.head
46            while last_node.next:
47                last_node = last_node.next
48
49            last_node.next = new_node
50
51        def display(self):
52            """
53            Displays the contents of the linked list in a readable format.
54            """
55            current_node = self.head
56            while current_node:
57                print(current_node.data, end=' -> ')
58                current_node = current_node.next
59            print("None")
60    if __name__ == "__main__":
61        # Example usage of the LinkedList class
62        linked_list = LinkedList()
63        linked_list.insert(10)
64        linked_list.insert(20)
65        linked_list.insert(30)
66        print("Linked List contents:")
67        linked_list.display()
68
```

OUTPUT:

```
Linked List contents:  
10 -> 20 -> 30 -> None  
PS C:\Users\sathw>
```

Task 3 – Linked List

Justification:

Linked List allows dynamic memory allocation and easy insertion or deletion. It is useful when the size of data changes frequently.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:  
  
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

PROMPT:

#Create a Python Binary Search Tree with recursive insert and in-order traversal methods, maintaining BST properties, with docstrings, comments, and example usage.

```
C:\Users\sathw> 11.1.1.py > ...
1 class BST:
2     """Binary Search Tree Implementation"""
3
4     def __init__(self, value):
5         self.value = value
6         self.left = None
7         self.right = None
8
9     def insert(self, value):
10        """Recursive insert."""
11        if value < self.value:
12            if self.left:
13                self.left.insert(value)
14            else:
15                self.left = BST(value)
16        else:
17            if self.right:
18                self.right.insert(value)
19            else:
20                self.right = BST(value)
21
22    def inorder(self):
23        """In-order traversal (sorted output)."""
24        if self.left:
25            self.left.inorder()
26        print(self.value, end=" ")
27        if self.right:
28            self.right.inorder()
29
30    # ----- MAIN PROGRAM -----
31    # Take input from user
32    values = list(map(int, input("Enter numbers separated by space: ").split()))
33    # Create BST with first value
34    root = BST(values[0])
35    # Insert remaining values
36    for value in values[1:]:
37        root.insert(value)
38    # Display sorted output
39    print("In-order Traversal (Sorted):")
40    root.inorder()
41
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
025.18.0-win32-x64\bundled\libs\debugpy\launcher '60960' '--' 'C:\Users\sathw\11.1.1.py'
Enter numbers separated by space: 50 70 20 40 10 15 35
Enter numbers separated by space: 50 70 20 40 10 15 35
In-order Traversal (Sorted):
In-order Traversal (Sorted):
10 15 20 35 40 50 70
PS C:\Users\sathw>
```

Task 4 – Binary Search Tree (BST)

Justification:

BST maintains elements in sorted order for efficient searching. It provides faster insert and search operations compared to linear structures.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

PROMPT:

#Implement a Python Hash Table using chaining for collision handling with insert, search, and delete methods, custom hash function, proper documentation, and example usage.

```
C:\Users> sathw > 11.1.1.py > ...
1 class HashTable:
2     def __init__(self, size=10):
3         self.size = size
4         self.table = [[] for _ in range(size)]
5
6     def _hash(self, key):
7         return hash(key) % self.size
8
9     def insert(self, key, value):
10        index = self._hash(key)
11        for i, (k, v) in enumerate(self.table[index]):
12            if k == key:
13                self.table[index][i] = (key, value)
14                print("Key updated.")
15                return
16        self.table[index].append((key, value))
17        print("Inserted successfully.")
18
19    def search(self, key):
20        index = self._hash(key)
21        for k, v in self.table[index]:
22            if k == key:
23                return v
24        return None
25
26    def delete(self, key):
27        index = self._hash(key)
28        for i, (k, v) in enumerate(self.table[index]):
29            if k == key:
30                del self.table[index][i]
31                print("Deleted successfully.")
32                return
33        print("Key not found.")
34
35    def display(self):
36        for i, bucket in enumerate(self.table):
37            print(f"Index {i}: {bucket}")
38
39
```

```

39
40 ht = HashTable()
41
42 while True:
43     print("\n1.Insert 2.Search 3.Delete 4.Display 5.Exit")
44     choice = input("Enter choice: ")
45
46     if choice == "1":
47         key = input("Enter key: ")
48         value = input("Enter value: ")
49         ht.insert(key, value)
50
51     elif choice == "2":
52         key = input("Enter key to search: ")
53         result = ht.search(key)
54         print("Value:", result if result else "Not found")
55
56     elif choice == "3":
57         key = input("Enter key to delete: ")
58         ht.delete(key)
59
60     elif choice == "4":
61         ht.display()
62
63     elif choice == "5":
64         break
65
66
67

```

OUTPUT:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 1
Enter key: 101
Enter value: BALA
Inserted successfully.

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 2
Enter key to search: 1
Value: Not found

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 2
Enter key to search: 101
Value: BALA

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 3
Enter key to delete: 101
Deleted successfully.

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 4
Index 0: []
Index 1: []
Index 2: []
Index 3: []
Index 4: []
Index 5: []
Index 6: []
Index 7: []
Index 8: []
Index 9: []

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Enter choice: 5
PS C:\Users\sathw> 

```

Task 5 – Hash Table

Justification:

Hash Table provides very fast lookup, insertion, and deletion using keys. It is ideal for applications requiring quick data retrieval.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
    pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

PROMPT:

#Generate a Python Graph implementation using an adjacency list with add_vertex, add_edge, and display methods, including documentation, comments, and example usage.

```

C:\Users> sathw > 11.16.py > ...
1  #Generate a Python Graph Implementation using an adjacency list with add_vertex, add_edge, and display methods, including documentation, comments, and example usage.
2  class Graph:
3      """
4      A class to represent a graph using an adjacency list.
5
6      Attributes:
7      graph (dict): A dictionary to store the graph where keys are vertices and values are lists of adjacent vertices.
8
9      Methods:
10     add_vertex(v): Adds a vertex to the graph.
11     add_edge(v1, v2): Adds an edge between two vertices in the graph.
12     display(): Displays the graph in an adjacency list format.
13     """
14
15     def __init__(self):
16         """Initialize the graph as an empty dictionary."""
17         self.graph = {}
18
19     def add_vertex(self, v):
20         """
21         Add a vertex to the graph.
22
23         Parameters:
24         v (str): The vertex to be added.
25         """
26         if v not in self.graph:
27             self.graph[v] = []
28
29     def add_edge(self, v1, v2):
30         """
31         Add an edge between two vertices in the graph.
32
33         Parameters:
34         v1 (str): The first vertex.
35         v2 (str): The second vertex.
36         """
37         self.graph.setdefault(v1, []).append(v2)
38         self.graph.setdefault(v2, []).append(v1)
39

```

```

39
40     def display(self):
41         """Display the graph in an adjacency list format."""
42         for vertex in self.graph:
43             print(vertex, "->", self.graph[vertex])
44
45     # Example usage
46     if __name__ == "__main__":
47         g = Graph()
48
49         while True:
50             print("\n1.Add Vertex 2.Add Edge 3.Display 4.Exit")
51             choice = input("Enter choice: ")
52
53             if choice == "1":
54                 v = input("Enter vertex: ")
55                 g.add_vertex(v)
56
57             elif choice == "2":
58                 v1 = input("Enter first vertex: ")
59                 v2 = input("Enter second vertex: ")
60                 g.add_edge(v1, v2)
61
62             elif choice == "3":
63                 g.display()
64
65             elif choice == "4":
66                 break
67
68
69
70
71
72
73
74

```

OUTPUT:



```
1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 1
Enter vertex: A

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 1
Enter vertex: B

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 1
Enter vertex: C

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 2
Enter first vertex: A
Enter second vertex: B

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 2
Enter first vertex: A
Enter second vertex: C

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 3
A -> ['B', 'C']
B -> ['A']
C -> ['A']

1.Add Vertex 2.Add Edge 3.Display 4.Exit
Enter choice: 4
PS C:\Users\sathw>
```

Task 6 – Graph

Justification:

Graph represents relationships between nodes using edges. It is suitable for networks, routes, and connected systems.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
```

```
pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```

C:\Users> sathw > 11.1.6.py > ...
1  import heapq
2
3  class PriorityQueue:
4      """
5      Priority Queue using heapq.
6      Lower number = Higher priority.
7      """
8
9      def __init__(self):
10         self.elements = []
11
12     def enqueue(self, item, priority):
13         """Add item with priority."""
14         heapq.heappush(self.elements, (priority, item))
15         print("Item added successfully.")
16
17     def dequeue(self):
18         """Remove highest priority item."""
19         if not self.is_empty():
20             return heapq.heappop(self.elements)[1]
21         return None
22
23     def is_empty(self):
24         """Check if queue is empty."""
25         return len(self.elements) == 0
26
27     def display(self):
28         """Display queue contents."""
29         if self.is_empty():
30             print("Priority Queue is empty.")
31         else:
32             print("Priority Queue Contents:")
33             for priority, item in sorted(self.elements):
34                 print(f"Priority: {priority}, Item: {item}")
35
36 # ----- MAIN PROGRAM -----
37
38 if __name__ == "__main__":
39     pq = PriorityQueue() pq = <_main_.PriorityQueue object at 0x000002A2627EC830>
40
41     while True:
42         print("\n1.Enqueue 2.Dequeue 3.Display 4.Exit")
43         choice = input("Enter your choice: ")
44
45

```

```

C:\Users> sathw > 11.1.6.py > ...
36
37 # ----- MAIN PROGRAM -----
38
39 if __name__ == "__main__":
40     pq = PriorityQueue() pq = <_main_.PriorityQueue object at 0x000002A2627EC830>
41
42     while True:
43         print("\n1.Enqueue 2.Dequeue 3.Display 4.Exit")
44         choice = input("Enter your choice: ")
45
46         if choice == "1": choice = '1'
47         item = input("Enter item: ") item = 'Assignment'
48         priority = int(input("Enter priority (lower number = higher priority): "))
49         pq.enqueue(item, priority)
50
51         elif choice == "2":
52             removed = pq.dequeue()
53             if removed:
54                 print("Removed:", removed)
55             else:
56                 print("Priority Queue is empty.")
57
58         elif choice == "3":
59             pq.display()
60
61         elif choice == "4":
62             print("Exiting program...")
63             break
64
65         else:
66             print("Invalid choice. Try again.")
67
68
69
70
71
72
73
74
75
76
77

```

OUTPUT:

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 1
Enter item: Assignment
Enter priority (lower number = higher priority): 2
Item added successfully.
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 1
Enter item: Exam
Enter priority (lower number = higher priority): 1
Item added successfully.
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 3
Priority Queue Contents:
Priority: 1, Item: Exam
Priority: 2, Item: Assignment
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 3
Priority Queue Contents:
Priority: 1, Item: Exam
Priority: 2, Item: Assignment
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Priority Queue Contents:
Priority: 1, Item: Exam
Priority: 2, Item: Assignment
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Priority: 1, Item: Exam
Priority: 2, Item: Assignment
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Priority: 2, Item: Assignment
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
```

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 2
Enter your choice: 2
Removed: Exam
```

```

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 3
Priority Queue Contents:
Removed: Exam

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 3
Priority Queue Contents:
Priority: 2, Item: Assignment

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 4
Priority: 2, Item: Assignment

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 4
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter your choice: 4
Exiting program...
PS C:\Users\sathw> 

```

Task 7 – Priority Queue

Justification:

Priority Queue processes elements based on priority instead of order of arrival. It is used in scheduling and shortest path algorithms.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
```

```
pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

PROMPT:

#Implement a Python Deque using collections.deque with insert_front, insert_rear, remove_front, and remove_rear methods, including proper documentation and example usage.

```
C:\Users\sathw> 11.1.1.py> ...
1  #Implement a Python Deque using collections.deque with insert_front, insert_rear, remove_front, and remove_rear methods, including proper documentation and example
2  from collections import deque
3  class Deque:
4      """
5      A double-ended queue (deque) implementation using collections.deque.
6
7      Methods:
8      - insert_front(item): Inserts an item at the front of the deque.
9      - insert_rear(item): Inserts an item at the rear of the deque.
10     - remove_front(): Removes and returns the item at the front of the deque.
11     - remove_rear(): Removes and returns the item at the rear of the deque.
12     """
13
14     def __init__(self):
15         """Initialize an empty deque."""
16         self.deque = deque()
17
18     def insert_front(self, item):
19         """Insert an item at the front of the deque."""
20         self.deque.appendleft(item)
21
22     def insert_rear(self, item):
23         """Insert an item at the rear of the deque."""
24         self.deque.append(item)
25
26     def remove_front(self):
27         """Remove and return the item at the front of the deque. Raises IndexError if empty."""
28         if not self.deque:
29             raise IndexError("Deque is empty")
30         return self.deque.popleft()
31
32     def remove_rear(self):
33         """Remove and return the item at the rear of the deque. Raises IndexError if empty."""
34         if not self.deque:
35             raise IndexError("Deque is empty")
36         return self.deque.pop()
37
38
39 if __name__ == "__main__":
40     # Example usage of the Deque class
41     my_deque = Deque()
42
43     my_deque.insert_rear(1)
44     my_deque.insert_rear(2)
45     my_deque.insert_front(0)
46
47     print("Deque after insertions:", list(my_deque.deque))
48
49     print("Removed from front:", my_deque.remove_front())
50     print("Removed from rear:", my_deque.remove_rear())
51
52     print("Deque after removals:", list(my_deque.deque))
53
54
55
56
```

OUTPUT:

```
Deque after insertions: [0, 1, 2]
Removed from front: 0
Removed from rear: 2
Removed from front: 0
Removed from rear: 2
Removed from rear: 2
Deque after removals: [1]
Deque after removals: [1]
PS C:\Users\sathw>
```

Task 8 – Deque

Justification:

Deque allows insertion and deletion from both ends. It is flexible and can work as both stack and queue.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:

#Analyze the given campus system features, select the most appropriate data structure for each with justification, present the mapping in a table, and implement one feature as a documented Python program.

```

C:\Users>sathw > 11.1.1.py > ...
1 #Analyze the given campus system features, select the most appropriate data structure for each with justification, present the mapping in a table, and implement on.
2 class StudentRecords:
3     """
4     Student Records Management using Dictionary.
5     Key = Student ID
6     Value = Student details (name, age, major)
7     """
8
9     def __init__(self):
10         self.records = {}
11
12     def add_student(self, student_id, name, age, major):
13         """Add a new student."""
14         self.records[student_id] = {
15             "name": name,
16             "age": age,
17             "major": major
18         }
19         print("Student added successfully.")
20
21     def get_student(self, student_id):
22         """Retrieve student details."""
23         if student_id in self.records:
24             info = self.records[student_id]
25             print(f"ID: {student_id}")
26             print(f"Name: {info['name']}")
27             print(f"Age: {info['age']}")
28             print(f"Major: {info['major']}")
29         else:
30             print("Student not found.")
31
32     def remove_student(self, student_id):
33         """Remove student from records."""
34         if student_id in self.records:
35             del self.records[student_id]
36             print("Student removed successfully.")
37         else:
38             print("Student not found.")
39
40     def display_all_students(self):
41         """Display all student records."""
42         if not self.records:
43             print("No student records available.")
44         else:
45             for student_id, info in self.records.items():

```

```

44         else:
45             for student_id, info in self.records.items():
46                 print(f"ID: {student_id}")
47                 print(f"Name: {info['name']}")
48                 print(f"Age: {info['age']}")
49                 print(f"Major: {info['major']}")
50 # ----- MAIN PROGRAM -----
51 if __name__ == "__main__":
52     student_records = StudentRecords()
53     while True:
54         print("\n1. Add Student")
55         print("2. View Student")
56         print("3. Remove Student")
57         print("4. Display All Students")
58         print("5. Exit")
59         choice = input("Enter your choice: ")
60         if choice == "1":
61             student_id = input("Enter Student ID: ")
62             name = input("Enter Name: ")
63             age = input("Enter Age: ")
64             major = input("Enter Major: ")
65             student_records.add_student(student_id, name, age, major)
66
67         elif choice == "2":
68             student_id = input("Enter Student ID to search: ")
69             student_records.get_student(student_id)
70
71         elif choice == "3":
72             student_id = input("Enter Student ID to remove: ")
73             student_records.remove_student(student_id)
74
75         elif choice == "4":
76             student_records.display_all_students()
77
78         elif choice == "5":
79             print("Exiting program...")
80             break
81         else:
82             print("Invalid choice. Try again.")

```

OUTPUT:

```
1. Add Student
2. View Student
3. Remove Student
4. Display All Students
5. Exit
Enter your choice: 1
Enter Student ID: 10B1
Enter Name: BALA
Enter Age: 21
Enter Major: CSE
Student added successfully.
```

```
1. Add Student
2. View Student
3. Remove Student
4. Display All Students
5. Exit
Enter your choice: 2
Enter Student ID to search: 10B1
ID: 10B1
Name: BALA
Age: 21
Major: CSE
```

```
1. Add Student
2. View Student
3. Remove Student
4. Display All Students
5. Exit
Enter your choice: 3
Enter Student ID to remove: 10B1
Student removed successfully.
```

```
1. Add Student
2. View Student
3. Remove Student
4. Display All Students
5. Exit
Enter your choice: 4
No student records available.
```

```
1. Add Student
2. View Student
3. Remove Student
4. Display All Students
5. Exit
Enter your choice: 5
Exiting program...
PS C:\Users\sathw> █
```

JUSTIFICATION TASK -9

1 Student Records – Dictionary

Stores data using unique ID for fast access.

2 Course Enrollment – List

Allows dynamic addition and removal of courses.

3 Faculty Information – Dictionary

Provides quick lookup using Faculty ID.

4 Campus Events – List

Stores multiple events and supports easy updates.

Task Description #10: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:

#Generate a menu-driven Python program for an Order Processing System using a Queue (FIFO) with user input support in VS Code, including place_order, process_order, and display_orders methods, proper documentation, and continuous loop until exit.

```

C:\Users> sathw > 11.1.py > ...
1 #Generate a menu-driven Python program for an Order Processing System using a Queue (FIFO) with user input support in VS Code, including place_order, process_order
2 from collections import deque
3 class OrderProcessingSystem:
4     def __init__(self):
5         """Initialize the order processing system with an empty queue."""
6         self.orders = deque()
7
8     def place_order(self, order: str) -> None:
9         """
10         Place a new order in the queue.
11
12         Args:
13             order (str): The order to be placed.
14         """
15         self.orders.append(order)
16         print(f"Order '{order}' has been placed.")
17
18     def process_order(self) -> None:
19         """
20         Process the next order in the queue.
21
22         Raises:
23             IndexError: If there are no orders to process.
24         """
25         if not self.orders:
26             print("No orders to process.")
27             return
28         order = self.orders.popleft()
29         print(f"Order '{order}' has been processed.")
30
31     def display_orders(self) -> None:
32         """Display all pending orders in the queue."""
33         if not self.orders:
34             print("No pending orders.")
35             return
36         print("Pending Orders:")

```

```

35         return
36         print("Pending Orders:")
37         for idx, order in enumerate(self.orders, start=1):
38             print(f"{idx}. {order}")
39 def main():
40     system = OrderProcessingSystem()
41
42     while True:
43         print("\nOrder Processing System Menu:")
44         print("1. Place Order")
45         print("2. Process Order")
46         print("3. Display Orders")
47         print("4. Exit")
48
49         choice = input("Enter your choice (1-4): ")
50
51         if choice == '1':
52             order = input("Enter the order details: ")
53             system.place_order(order)
54         elif choice == '2':
55             system.process_order()
56         elif choice == '3':
57             system.display_orders()
58         elif choice == '4':
59             print("Exiting the Order Processing System. Goodbye!")
60             break
61         else:
62             print("Invalid choice. Please try again.")
63 if __name__ == "__main__":
64     main()
65

```

OUTPUT:

025.16.0-win32-x64 (bundled) (lib5,debugpy (launcher) 52/93 -- C:\Users\sathw\11.1.1.py

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 1
Enter the order details: 0101
Order '0101' has been placed.

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 1
Enter the order details: 0101
Order '0101' has been placed.

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 0102
Invalid choice. Please try again.

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 0102
Invalid choice. Please try again.

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 0102
Invalid choice. Please try again.

Python Deb...

Python Deb...

Python Deb...

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

2. Process Order
3. Display Orders
4. Exit
4. Exit

Enter your choice (1-4): 3

Enter your choice (1-4): 3

Pending Orders:

1. 0101
2. 0101

Pending Orders:

1. 0101
2. 0101

Order Processing System Menu:

2. 0101

Order Processing System Menu:

Order Processing System Menu:

1. Place Order
2. Process Order
3. Display Orders
4. Exit

Enter your choice (1-4): 4

Exiting the Order Processing System. Goodbye!

PS C:\Users\sathw> █

JUSTIFICATION:

Task 10 – Data Structure Mapping

1 Shopping Cart – Linked List

Allows dynamic add/remove of products easily.

2 Order Processing – Queue

Processes orders in FIFO order.

3 Top-Selling Products – Priority Queue

Retrieves highest selling products first.

4 Product Search Engine – Hash Table

Provides fast lookup using product ID.

5 Delivery Route Planning – Graph

Represents warehouses and locations as connected nodes.